Faculty Systems Harness

Comprehensive Architecture, Compilation, Runtime, Workforce, Facilities, Labs and Governance

Architecture working draft | 26 October 2026 | For design review and implementation specification

Executive summary

Faculty Systems is a governed, review-led intelligence operating system. It maintains evidence-backed Positions, conducts recurring Reviews, compiles and evaluates Methods, Lines and Moves, coordinates authorized work through AI Workers and Agents, and learns from recorded outcomes. The shared C# Harness supplies execution, persistence, evaluation, governance, pooling, caching, and ledgers. Domain-specific Faculties and Facilities supply definitions and qualified repertoires.

The central architecture is deliberately not an LLM-centric agent loop. AI Workers are scoped mini-Faculties; Agents are job/case coordinators and bounded executors; Skills are typed capabilities, some of which may invoke LLMs or JEV. A Faculty can operate in independent review-only mode without taking responsibility for planning or execution.

1. Design principles and decisions

- Position and Review are the cognitive center; they are native harness subsystems, not optional reporting layers.
- One reusable Faculty Harness supports multiple Faculties, AI Workers, Facilities and optionally federated harness instances.
- The AI Worker is a mini-Faculty managing an enduring function or Position; an Agent manages bounded jobs and cases.
- Runtime definitions and commercial offerings are separate: a Desk, Review, Agent, Worker, Facility or Program may be packaged independently.
- Skills are executable units; Moves carry intent and expected Position effects; Lines represent conditional strategic progressions.
- Rubrics judge, Gates govern, Policies constrain, Playbooks organize alternatives, and the Compiler creates versioned executable/evaluable compositions.
- Every consequential event is ledgered, but caches and projections are not authoritative.
- Operational learning can be automatic within approved bounds; method qualification, authority expansion and production promotion require governance.
- Formal field-method distillation is reserved for governed planning and execution/experimentation, not review-only observation.

2. Architectural taxonomy

Primitive	Meaning	Example
Faculty	Independent domain of mastery, objectives, Positions, Reviews and methods	Kozmo Commercial Faculty
Facility	Deployable package of domain capabilities, not another Faculty	Renewal Facility
Area	Persistent scope of managed reality	Contracts Area

Repository	Governed artifacts, evidence and records	Contract Repository
Desk	Customer-facing intake, cases and SLAs	Contract Desk
Service	Purchasable bounded deliverable	Independent Contract Review
Program	Ongoing objective and managed initiative	Renewal Advantage
AI Worker	Mini-Faculty with enduring Position stewardship	Contract Manager
Agent	Bounded job, coordination or execution capability	Invoice Dispute Agent
Skill	Typed reusable executable capability	Extract clause / JEV assess risk
Position	Living evidence-backed state of a subject	Renewal Position
Review	Versioned independent reassessment	Renewal Readiness Review
Method / Line / Move	Know-how / progression / intentional intervention	Validation Bridge
Policy / Gate / Rubric	Constraint / decision point / assessment	Approval / authority / continuity
Ledger	Append-only causal record	Review and outcome events

3. Harness architecture

The logical Harness comprises Reality, Intelligence, Agency, Work, Information, Runtime, Governance and Learning modules. A modular monolith is the recommended initial deployment; module boundaries and contracts permit later independent scaling.

Module	Responsibilities
Reality	Evidence, research, claims, Positions, snapshots, deltas, Reviews
Intelligence	Compiler, JEV adapters, rubrics, Vichara, simulations, playbooks, methods
Agency	Faculty Units, AI Workers, Agents, Skills, repertoire resolution
Work	Requests, Jobs, Tasks, Desks, cases, workflows, alerts, reminders, initiatives
Information	Repositories, artifacts, event ledgers, trace/provenance projections
Runtime	Queues, leases, worker pools, caches, scheduler, model routing, budgets, checkpoints
Governance	Mandates, policies, gates, entitlements, approvals, isolation
Learning	Outcome attribution, regression, experiments, promotion/restriction/retirement

4. Faculty, Mini-Faculty and Federation

A Faculty Unit is a governed intelligence unit exposing Position, Review, Job and capability contracts. Full Faculties and mini-Faculty AI Workers share these mechanisms without inheriting an entire separate runtime. A Commercial Manager may steward the full commercial portfolio, while Contract, Invoice and Relationship Managers steward narrower Positions. A specialized Agent may be independently purchased but need not steward a persistent functional Position.

A Federation Harness may coordinate independent Faculties such as Kozmo Commercial, Zotra Growth and Hipstraw Campaign. It maintains cross-Faculty Positions and Reviews, exchanges authorized evidence and coordinates dependencies. It never inherits or overrides constituent Faculty authority. Initially, multiple Faculties may share one process and infrastructure; later they can run in isolated deployments.

5. Facilities, Areas, Desks, Services and Programs

A Facility is a versioned capability assembly, not a new Faculty or a separate runtime. The Renewal Facility can package renewal Position schemas, Review definitions, JEV rubrics, policies, playbooks, compiled Lines, Skills and optional Agents/Workers. The Contracts Area organizes contracts as managed subjects. The Contract Repository preserves artifacts. The Contract Desk accepts cases. The Independent Contract Review Service is the purchasable deliverable. A Renewal Advantage Program is a recurring engagement using the same Facility.

Offering	Facility	Mode	Execution ownership
Independent Contract Review	Contract	Review-only	No
Contract Desk	Contract	Intake and case service	Only explicitly entitled/authorized work
Contract Manager	Contract	Mini-Faculty Worker	Mandate-bound
Independent Renewal Review	Renewal	Review-only	No
Renewal Advantage Program	Renewal	Recurring review + optional management	Separately authorized

6. Positions and Reviews

A Position is a persistent identity with immutable versioned snapshots and deltas, not a mutable scorecard. Each dimension links to evidence, uncertainty, incentives, objectives, constraints, and observed execution status. Reviews can examine a Position regardless of whether the Faculty, humans or third parties are executing the relevant work.

A ReviewDefinition specifies scope, evidence requirements, rubrics, evaluation stages, materiality and outputs. ReviewRun pins a Position snapshot and definition versions, records judgments, findings, unresolved questions, and next triggers. New evidence yields a PositionDelta and possibly a focused re-Review. Portfolio Reviews may cross contracts, relationships, financial, insurance, liability, confidentiality, compliance, reliability, risk, projections and opportunities.

7. Compilation grammar

The compact grammar is Method, Line, Move and composed Skills, with Protocol as the execution/governance contract. There is no mandatory separate ActionDefinition. The compiled intermediate representation is a typed graph with Skill, Rubric, Gate, Branch, Join and Checkpoint nodes.

Element	Role	Boundary
Skill	Typed execution capability; deterministic, JEV, LLM or external API	No independent strategic intent
Move	Intentional inquiry or intervention with applicability, evidence, risk, expected effects	Can compose multiple Skills

Line	Conditional progression of Moves with reassessment and countermoves	Not identical to workflow
Method	Reusable approach across suitable Positions	May offer alternative Lines
Protocol	Entry/exit, retries, checkpoints, failure and governance rules	Does not duplicate Line graph
Playbook	Situation-oriented set of candidate/qualified approaches	Not automatically executable
Rubric	Typed evaluation question, scales, evidence and evaluator	JEV may implement judgments
Gate	Pass/fail/defer/escalate enforcement point	Must fail closed where authority is missing
Policy	Persistent scoped constraint and precedence	Cannot be overridden by model prose
Position model	Bands, quadrants, archetypes, transitions, uncertainty	Classification narrows options, not authority

8. JEV and evaluation

JEV is a bounded judgment adapter used by Reviews, compiler qualification, skill validation, alert assessment and Labs. Each judgment is identified by rubric version, normalized evidence snapshot, permitted outputs, evaluator configuration and provenance. Choice, Score and Noul results are typed inputs to deterministic policies, not self-authorizing decisions. Missing evidence and model uncertainty are separately represented.

Compiler evaluation includes applicability, evidence sufficiency, policy compliance, expected value, downside risk, optionality, scenario robustness, execution feasibility, and qualification state. Lab-qualified candidate methods are not described as field-proven.

9. Requests, Jobs, Tasks and workflows

A Request records demand and intent; triage can reject, defer, clarify or accept it. An accepted Job is an accountable work commitment with owner, mandate, outcome criteria, limits and protocol. A Task is a bounded unit within the compiled Job graph. A WorkflowInstance coordinates dependencies, Skills, Moves, approvals, waiting, retries and checkpoints. Jobs may also arise from alerts, scheduled Reviews, workers, policies or Lab experiments.

DeskCase is the customer-visible service lifecycle; it is not identical to a Job. One case may create multiple Jobs; a Job can support several authorized requests. Completion of a Job does not automatically prove the originating Request was fulfilled.

10. Agents and AI Workers

An AI Worker is a mini-Faculty: a persistent logical manager of a function or Position with delegated objectives, recurring Reviews, a repertoire, agents and ledgers. An Agent is a bounded work capability. Recommended role profiles are Case Executive (case/job lifecycle), Coordination (dependencies and handoffs), Task (work package execution) and Observation/Investigation (bounded evidence acquisition). They share one runtime and do not require direct LLM access.

Commercial offerings may sell specialized Agents independently. A Renewal Negotiation Agent can operate on a case Position and invoke specialized Skills without becoming a full Relationship Manager. New Worker

types should reflect enduring managerial responsibility, but the commercial catalog can contain many specialized Agent offerings.

11. Intelligent alerts, reminders, campaigns and managed objects

Alerts are durable findings with evidence, severity, acknowledgment, escalation and disposition; AlertService detects and reassesses them using deterministic conditions, Position deltas and optional JEV rubrics. Reminders are scheduled obligations or notifications. Campaigns, Projects and Programs are managed initiatives with objectives, Positions, reviews, milestones and outcomes. None needs to be a standalone Agent; Agents may service them.

12. Runtime resource management

Logical Agents and Workers persist as definitions and durable state; compute is acquired through bounded execution leases on demand. The runtime should provide job queues, priorities, tenant fairness, model/JEV quotas, scheduler, concurrency, budgets, checkpoints, retry/backoff, idempotency, and telemetry. Pools allocate stateless or resettable execution capacity rather than permanently running conversational LLM agents.

Cache definitions, source-derived evidence, immutable Position read projections, bounded contexts, JEV results and compiled artifacts only with versioned keys and source-freshness constraints. Never treat a cache as the authoritative ledger, and always recheck effectful authorization at dispatch.

13. Ledgers, repositories and learning

Use an append-only event store with typed projections for evidence, Positions, Reviews, methods, decisions, execution, outcomes, experiments and commercial usage. Events include tenant, actor, correlation/causation, event and observation time, schema version, provenance, and relevant definition versions. External effects require intent, approval, dispatch, confirmation and reconciliation records.

The Learning Engine analyzes Skill quality, Move effects, Line effectiveness, protocol performance, Agent/Worker productivity, rubric calibration and runtime cost. It proposes changes and routes substantive experimental questions to Labs. Automatic changes are confined to bounded operational policy; promotion, authority expansion, and field-method distillation require governed decisions.

14. Optional Faculty Labs

Each Faculty may attach a Lab using the same Harness primitives but isolated experimental authority, compute profiles, repositories, positions and ledgers. A Lab Manager AI Worker can oversee experiments through research, simulation, evaluation and compiler Agents. Shared Blue Salt Labs investigate general competence and harness methods; Faculty Labs investigate domain methods; customer-scoped Labs isolate proprietary data. Simulated success does not constitute field validation.

15. Commercial packaging and entitlements

CommercialOffering is distinct from runtime object identity. Offerings can be Reviews, Desks, specialized Agents, AI Workers, Programs, Facilities or Faculty subscriptions. Charge by subscription, case, review, volume or carefully attributable outcomes; Skills are usually metered internally. Entitlement to a product does not confer authority to perform external actions. Usage ledger records billable units separately from evidence and execution history.

16. Knowledge and package format

Author doctrine, rationales and examples in Markdown; authoritative executable definitions in schema-validated YAML/JSON; compile immutable signed Faculty packages containing versioned Skills, policies, gates, rubrics, repertoire and qualification metadata. The C# Harness consumes compiled packages rather than interpreting arbitrary Markdown as executable instructions.

17. Worked reference scenario: SaaS renewal

Fictional example: $120,000 annual subscription, $138,000 proposed renewal, 80 of 200 seats observed active, 45 days until renewal, 30-day nominal notice requirement and unverified 90-day migration estimate.

Stage	Artifact	Enforcement
Intake	Renewal Desk Request and Case	Entitlement and review-only mandate
Evidence	Contract, proposal, usage, assumptions	Source verification and provenance
Position	Renewal Position v1	Immutable snapshot with unknowns
Review	Economic, continuity, optionality findings	Versioned rubrics and JEV judgments
Compilation	Validation Bridge candidate Line	Applicability, policy and qualification gates
Decision	Explore bridge; no external send	Authority gate blocks execution
New evidence	Supplier rejects bridge	Position v2 and re-Review
Authorized extension	Separate planning/execution mandate	Worker assignment, approved Moves and Skills
Outcome	Observed response and actual terms	Ledger, attribution and learning

18. Proposed C# project boundaries

FacultySystems.Contracts; FacultySystems.Reality; FacultySystems.Methods; FacultySystems.Governance; FacultySystems.Work; FacultySystems.Agency; FacultySystems.Runtime; FacultySystems.Information; FacultySystems.Learning; FacultySystems.Labs; FacultySystems.Federation; FacultySystems.Adapters; FacultySystems.Tests. Start with a modular monolith and explicit interfaces; avoid prematurely creating microservices.

19. Implementation phases and acceptance gates

Phase	Scope	Acceptance evidence
1	Contracts, mandates, evidence, Positions, ledger	Reconstructable initial renewal Position
2	Review, JEV fixtures, policies/gates	Review-only execution prohibited
3	Compiler, Moves, Lines, Skills graph	Typed, versioned candidate Line with tests
4	Requests, Jobs, Desk, Agents, queues, pools	Durable job execution and recovery
5	Workers, Facility packaging, commercial entitlements	Standalone Contract Review and managed Worker modes
6	Learning and optional Lab	Experimental evaluation, no unauthorized promotion
7	Federation	Cross-Faculty Review without authority leakage

20. Decisions to freeze and open design questions

Recommended to freeze

- One Harness with reusable services; logical Faculty Units and optional federation.
- AI Workers are mini-Faculty managers; Agents manage bounded jobs and specialist services.
- Facilities assemble capabilities; they do not automatically create Faculties.
- Review-only and managed execution are separate mandates.
- Skills are the executable primitives; Moves and Lines carry semantics, evaluation and progression.
- Policies, Gates and Rubrics remain independent typed objects.
- Ledgers are authoritative; learning and qualification are governed.

Open for specification

- Precise Faculty Unit interface, parent-child authority and cross-Faculty exchange contracts.
- Canonical Position schema, dimensional algebra and review consistency semantics.
- Compiler IR schema, conditional graph language and bounded loop semantics.
- JEV rubric registry, calibration, failure modes and caching rules.
- Runtime durability choice, storage/event architecture, tenancy and security model.
- Commercial offering catalog, pricing units and usage attribution.
- Method qualification thresholds and experimental design standards.

21. Implementation instruction for coding platforms

Claude Code or another coding platform should implement only frozen contracts and acceptance fixtures. No self-invented agent harness, autonomous planner, unbounded tool loop, new authority model or hidden orchestration. Any ambiguous architecture must be reported for approval. Require deterministic tests, negative authorization tests, golden ledger fixtures, crash-recovery tests and architecture-diff review before acceptance.

End of working architecture document.

22. Tools and Adaptive Experiences (Architecture Update)

Update dated 26 October 2026. This section extends the existing architecture with a first-class interaction and experience layer. It is additive: Tools and Experiences do not replace Skills, Desks, Facilities, Positions, or the Faculty Harness.

22.1 Definitions and boundaries

Concept	Responsibility	Example
Skill	Typed executable capability; may invoke C#, JEV, LLM or external API	Calculate renewal exposure
Tool	Reusable interactive or programmatic instrument that may invoke Skills	Proposal comparison editor
Adaptive Experience	Context-aware, stateful assembly of Tools and projections for a user goal	Negotiation Workspace
Desk	Intake, cases, service visibility, SLA and routing	Renewal Desk
AI Worker	Accountable Mini-Faculty managing enduring Positions	Relationship Manager
Agent	Bounded case, coordination or task executor	Negotiation case agent
Facility	Versioned assembly of intelligence and experience capabilities	Renewal Facility

22.2 Experience design model

An ExperienceDefinition specifies subject types, permitted Tools, layout templates, contextual bindings, role-specific views, adaptation rules, interaction contracts, accessibility requirements and authority policies. An ExperienceInstance binds that definition to tenant, principal, mandate, Position snapshot, active case/job, and user session. Experience state is a projection over authoritative Position, Review, Request, Job, Decision, Alert and Ledger state—not a competing source of truth.

Experience adaptation is governed by explicit inputs: Position changes, Review findings, user role, authorization, work status, uncertainty, experiments and preferences. The Experience Engine chooses validated components and layouts; LLMs may generate explanatory content or candidate layouts but cannot invent executable privileges, bypass gates, or silently mutate underlying Positions.

22.3 Reference Experiences

Intelligent Inbox

Attention-oriented projection of Alerts, Requests, Reviews, Jobs, decisions, opportunities and deadlines. Prioritization uses typed urgency/materiality policies; items link to underlying authoritative objects. Supports triage, acknowledgment, assignment, escalation and disposition.

General Workspace

Contextual work environment with evidence explorer, Position board, Review findings, tasks, approvals, artifacts and outcomes. May be reused across Facilities.

Negotiation Workspace

Kozmo environment for commercial Position, counterpart incentives, proposal comparison, scenario evaluation, candidate Lines/Moves, approval, authorized communication and observed counterparty responses. Changes to Position trigger re-evaluation of applicable Lines.

Campaign Story Manager

Hipstraw experience for campaign narrative, audience, offer, hypotheses, creative variants, experiment results, channel performance and story coherence. A campaign remains a managed initiative, not a Tool or Agent.

Experience Story Manager

Environment for journey stages, touchpoints, expectations, feedback, friction, experience hypotheses and outcomes. Can be delivered as an Experience or overseen by an Experience Manager AI Worker when enduring management is purchased.

22.4 Facility and commercial composition

A Facility manifest may package ToolDefinitions and ExperienceDefinitions alongside Position models, Reviews, Rubrics, Gates, Policies, Playbooks, Lines, Moves and Skills. For example, a Renewal Facility can power a standalone Negotiation Workspace subscription, a Renewal Desk, a Renewal Review Service, a specialized Negotiation Agent, or a managed Renewal Program. The same Tools can be reused in multiple Experiences without creating new runtime classes.

Tool/Experience entitlements are distinct from action authorization. Buying a Negotiation Workspace does not grant permission to send a proposal or bind a customer. Usage accounting records underlying Skills and compute while commercial packaging can charge for workspace subscriptions, seats, cases, managed services or bundles.

22.5 C# contracts and runtime services

Proposed contracts: ToolDefinition, ToolInstance (when stateful), ExperienceDefinition, ExperienceInstance, ExperienceState, ExperienceEvent, ExperienceComponentDefinition, ExperienceBinding, AdaptationPolicy, InteractionCommand; interfaces IToolRegistry, IExperienceEngine, IExperienceStateStore and IInteractionCommandDispatcher.

The Experience Engine composes and refreshes read models. All commands pass through a typed dispatcher that checks current entitlement, mandate, authority, policy and expected Position version before invoking a Skill, creating a Request/Job, or requesting approval. Runtime needs component caching, invalidation by ledger/Position version, event subscriptions, optimistic concurrency, accessibility, localization and telemetry.

22.6 Experience-specific ledgering and learning

Record ExperienceOpened, ComponentPresented, InteractionRequested, CommandAuthorized/Denied, ToolInvoked, DecisionPresented, DecisionRecorded, ExperienceRefreshed and ExperienceClosed where

material, with correlation to Position, Review, Job and Method versions. Avoid logging sensitive free-text or raw user content unnecessarily. Measure task completion, decision quality, clarity, time-to-resolution, user corrections, abandonment, accessibility, and business outcomes. The Learning Engine can propose better layouts, routing or Tool combinations; consequential changes require evaluation and governance.

22.7 Acceptance tests for the first vertical slice

- Renewal Inbox displays an evidence-linked alert and opens the correct Renewal Case and Position snapshot.
- Negotiation Workspace renders candidate Lines with evidence and qualification status; rejected Lines are not presented as executable.
- Supplier response produces a Position Delta and refreshes the workspace without overwriting historical Review state.
- Review-only mandate blocks supplier communication even if the Tool is visible or a premium Experience is entitled.
- Expired approval or stale Position version prevents effectful Skill dispatch.
- Experience state can be rebuilt from authoritative projections after cache loss or process restart.
- Accessibility, user-role differences, tenant isolation and component version compatibility pass regression tests.

22.8 Updated architectural decision

Freeze Experience as a first-class Harness category and Tool as a reusable interaction instrument. Preserve Skill as the executable primitive and Position/Review/Ledger as authoritative intelligence state. Experiences are adaptive views and interaction surfaces—not autonomous agents or parallel business databases.